

TolTransform

Kinematic Tolerance Analysis and Inverse Allocation: Theory and User Manual

July 2026

Table of Contents

Preface: What TolTransform Is.....	3
Part 1: Theoretical Foundations.....	4
1.1 Rigid-Body Transforms.....	4
1.2 The Linearized Model of a Tolerance.....	4
1.3 Kinematic Graphs.....	6
1.4 The Adjoint Representation and First-Order Sensitivity.....	7
1.5 Exact Propagation by Simulation.....	8
1.6 Statistical Characterization of the Propagated Error.....	9
1.7 Inverse Tolerance Allocation.....	11
1.8 Scope: What TolTransform Models, and What It Does Not.....	13
Part 2: Validation Against Three Physical Benchmarks.....	15
2.1 The Linear Stack-Up Benchmark.....	15
2.2 The Sine-Bar Lever-Arm Benchmark.....	15
2.3 The Common-Ancestor Cancellation Benchmark.....	16
Part 3: User Guide (Graphical User Interface).....	18
3.1 Overall Layout.....	18
3.2 Graph Editor: Building the Model.....	18
3.3 Tolerance Editor: Setting Per-DoF Tolerances.....	19
3.4 Run Panel: Configuring and Executing a Simulation.....	20
3.5 Results Viewer: Reading the Output.....	20
3.6 Point-Pair Analysis Panel.....	21
3.7 3D Frame Viewer (View → 3D Frame Viewer, Ctrl+3).....	21
Part 4: Brief Use-Case Examples.....	23
4.1 single_chain_fk_example.py: Basic Forward Verification.....	23
4.2 multi_chain_shared_frame_example.py: Shared-Frame Cancellation.....	23
4.3 allocation_example.py: The Inverse Allocator End-to-End.....	23
4.4 pareto_sensitivity_example.py: Sensitivity Ranking, Component Selection, and Reporting.....	24
Appendix A: Quick-Reference Glossary.....	25
Appendix B: Summary of Modeling Choices.....	26
Appendix C: Why Small Rotations Are Order-Agnostic.....	27
Appendix D: The Adjoint Matrix and the Sensitivity Jacobian.....	30

Appendix E: The Log-Barrier Allocation, Convexity, and the KKT Conditions.....	33
Appendix F: Exact Cancellation of Shared-Reference Error.....	35
Appendix G: Deriving the Statistical Summaries.....	37

Preface: What TolTransform Is

TolTransform lets a mechanical or optical engineer model an assembly as a graph of coordinate frames connected by toleranced rigid transforms. From that model it can propagate tolerances forward to find the resulting uncertainty at any point, or work backward from a target uncertainty to find how loose each interface is allowed to be.

In everyday terms: you describe how each part of an assembly is nominally positioned relative to the next, and how much play each of those joints is allowed to have, independently in each of six degrees of freedom. The tool then answers two complementary questions:

- **Forward verification (“FK”).** Given the tolerances specified everywhere, how much positional and angular error shows up at a given point, in the worst case or statistically?
- **Inverse allocation (“IK”).** Given a target error budget at some point, how loose can each individual interface’s tolerance be while still guaranteeing that budget?

A terminology note. Forward verification is labeled *FK* and inverse allocation is labeled *IK* throughout, by analogy with robotics’ forward and inverse kinematics. This is not classical inverse kinematics. “IK” here means inverse *tolerance allocation*: solving for per-interface tolerance bounds that achieve a target uncertainty envelope, not for a specific joint angle or pose.

This manual has four parts. **Part 1** develops the underlying theory: how a rigid-body tolerance is represented, how tolerances propagate through a kinematic chain, and how the inverse allocation problem is posed and solved as a convex optimization. **Part 2** describes three physical benchmarks that validate the theory against classical analytical results. **Part 3** is a practical guide to the desktop application. **Part 4** walks through worked examples. If you want to *use* the software, start at Part 3 and refer back to Part 1 as needed for the ideas behind each control. If you want the theory on its own, Parts 1 and 2 stand alone.

Part 1: Theoretical Foundations

This part builds the theory in dependency order. First, how a rigid-body transform is represented. Then how a *tolerance* on that transform is modeled as a small perturbation, and how such perturbations compose along a kinematic chain. Next, how a perturbation at one point in the chain relates, to first order, to its effect at another point, and how the exact (nonlinear) propagation is handled by simulation. Then how the resulting uncertainty is summarized statistically. Finally, how the whole apparatus is turned around to solve the inverse problem of *allocating* tolerances to meet a target.

1.1 Rigid-Body Transforms

A rigid-body transform between two coordinate frames is represented by a 4×4 homogeneous transformation matrix (HTM),

$$T = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix},$$

where $R \in SO(3)$ is a rotation matrix and $t \in \mathbb{R}^3$ is a translation. This is the standard representation of the group $SE(3)$ of rigid-body motions. Composing two transforms is ordinary matrix multiplication, $T_1 T_2$, and the inverse has the closed form

$$T^{-1} = \begin{bmatrix} R^T & -R^T t \\ 0^T & 1 \end{bmatrix},$$

which is cheaper and more numerically robust than a generic matrix inversion because it exploits the orthogonality of R .

A given rotation can be written down in several equivalent ways: a triple of Euler angles, a unit quaternion, or an axis-angle pair. These are just different coordinates for the same rotation, chosen for convenience. One subtlety deserves a warning, because it is a common source of error: rotations do not commute, so a triple of Euler angles means nothing until you also fix an *order* of composition (for instance, an intrinsic Z-Y-X sequence). This rotation-order ambiguity, along with a second, unrelated ambiguity in how a quaternion's four components are ordered, are two of the more common silent mistakes in any tool that accepts rotations from a user. Both must be pinned down by convention and stated openly.

1.2 The Linearized Model of a Tolerance

Think about what a toleranced interface physically is: a bolted flange, a dowel-pinned seat, a bearing fit. Each one can sit slightly off its nominal position in three directions, and slightly off its nominal orientation about three axes. Six small numbers describe the play completely:

$$\delta = (\delta_x, \delta_y, \delta_z, \theta_x, \theta_y, \theta_z).$$

That is the entire model of a tolerance in TolTransform: six independent degrees of freedom, each drawn from a probability distribution. A uniform distribution expresses a

hard worst-case bound (“anywhere within ± 0.05 mm”); a normal distribution expresses a statistical tolerance with a stated confidence level (“ ± 0.05 mm at 3σ ”).

There is a reason six plain numbers are allowed to stand in for something as nonlinear as a rigid-body pose. Full poses cannot be added together (the group $SE(3)$ of rigid motions is a curved space, not a vector space), but *small deviations* from a nominal pose live in its tangent space, which is flat, and a flat six-dimensional space is just a list of six numbers. In the robotics literature that tangent space is the Lie algebra $se(3)$, and a six-component object of this kind is called a **twist**, a term from screw theory, which treats translation and rotation as one six-dimensional quantity rather than two three-dimensional ones. The practical payoff of the formalism is the license to do ordinary linear algebra (adding, scaling, multiplying by matrices) on quantities that are secretly rotations.

From the algebra back to the group. To recover an actual rotation matrix from a small rotational twist $(\theta_x, \theta_y, \theta_z)$, use the exponential map of $SO(3)$, linearized to first order:

$$R \approx I + [\theta]_{\times}, [\theta]_{\times} = \begin{bmatrix} 0 & -\theta_z & \theta_y \\ \theta_z & 0 & -\theta_x \\ -\theta_y & \theta_x & 0 \end{bmatrix},$$

where $[\theta]_{\times}$ is the cross-product (skew-symmetric) matrix of the vector θ . This first-order approximation is what makes the whole framework tractable. It amounts to assuming that tolerances are small enough for rotations to commute and superpose linearly. If a tolerance is not small in this sense, say a loose joint with large angular play, then this linearized theory is the wrong tool for that interface, for the same reason any small-angle approximation fails once angles stop being small. (Appendix C derives the approximation, quantifies its error term, and proves the rotation-order independence it implies.)

A practical note: $I + [\theta]_{\times}$ is orthogonal only to first order, so it is good hygiene to re-project it onto the nearest true rotation matrix (in the Frobenius norm) before using it. The projection is easy to get from a singular value decomposition. Skip this step and small numerical errors will accumulate, over many compositions, into a matrix that is no longer a rotation at all.

Composing a perturbation onto a nominal transform. Given a nominal transform T_{nom} and a sampled twist δ , the perturbed transform is

$$T_{pert} = T_{nom} T_{\delta}(\delta),$$

where $T_{\delta}(\delta)$ is the small rigid motion built from δ as above. Composing *on the right* expresses the perturbation in the local (body-fixed) frame of the interface. That is the natural choice for a mechanical tolerance: a bolted joint’s angular play is naturally described relative to the joint’s own axes, not the world’s. It is a modeling choice, not a mathematical necessity. Composing on the left, in the world frame, would be equally valid for a different physical situation. But once fixed, the choice determines the exact form of every sensitivity relation in §1.4, and switching it later would mean re-deriving them all.

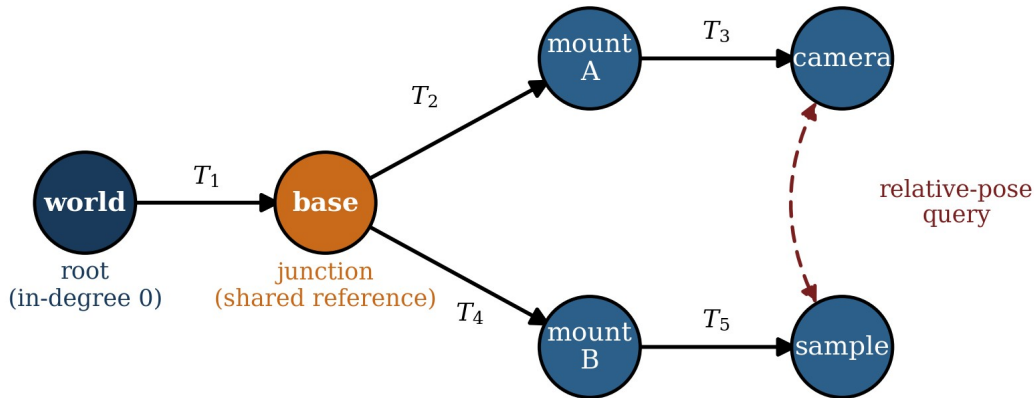
Two modeling assumptions bound what this theory can say:

Independence. No correlation is assumed between different degrees of freedom of the same interface, nor between different interfaces. Every one of the six components of every twist is drawn independently. This is the right level of fidelity for system-level error budgeting, where the goal is a defensible bound on end-to-end behavior. It would not be the right tool for a detailed manufacturing-process capability study of a single correlated feature.

A bound is not a lock. A tolerance can be marked as *fixed* for a particular calculation (for instance, when solving the inverse problem of §1.7 with some interfaces already chosen) without that meaning the interface has zero error. A fixed interface is still a real, nonzero source of uncertainty in the forward direction. “Fixed” only changes which variables an optimization is allowed to adjust.

1.3 Kinematic Graphs

An assembly is modeled as a directed graph: nodes are coordinate frames, and edges are toleranced rigid transforms between a parent and a child frame. Two structural restrictions hold throughout: the graph has no cycles, and no frame has more than one incoming edge. Together these say the assembly is a collection of **open, serial kinematic chains**, or trees in graph language, with no closed loops. This mirrors the classical distinction in mechanism design between open chains (a serial robot arm, unconstrained) and closed or parallel chains (a four-bar linkage, a Stewart platform). Closed chains need additional loop-closure constraint equations that this framework does not address.



A kinematic graph. Nodes are coordinate frames; directed edges are toleranced transforms. The root frame has no incoming edge. The junction frame is a shared reference from which two independent chains branch, which is what makes a relative-pose query between them well-posed.

A frame with more than one outgoing edge is a **shared reference**: an optical bench, say, to which two independent instrument chains are both bolted. This needs no special handling. It falls out naturally of modeling the assembly as a graph rather than as independent chains, and it is what makes it meaningful to ask about the *relative* tolerance between two components that sit on different chains but share a common ancestor. How much of a

shared reference’s own uncertainty cancels out of a relative measurement between two things attached to it is one of the central engineering questions this theory exists to answer. It is the subject of the benchmark in §2.3.

On disconnected sub-assemblies. If two components have no specified path of rigid connection between them, their relative tolerance is undefined, not zero. A tempting shortcut is to assume they share some common, perfectly rigid reference (“the world”) so that a relative measurement can always be computed. That is a modeling error, not a convenience. It silently injects a zero-tolerance connection that was never specified, which makes the error budget optimistic in exactly the case that matters most: two parts that are not, in fact, rigidly tied together. The correct treatment is to make the shared physical structure explicit as its own frame, with its own connecting tolerances, which may well be nonzero. A tool that refuses to answer an ill-posed relative-tolerance question is safer than one that silently answers it wrong.

1.4 The Adjoint Representation and First-Order Sensitivity

Every machinist knows the physical fact at the heart of this section: a tiny tilt at the spindle housing becomes a visible runout at the tool tip, because the tilt acts through the length of the tool. Wobble upstream is amplified downstream, and how much it is amplified depends on the geometry in between. Sensitivity analysis is the business of computing that amplification for every interface in the assembly at once: if one interface wiggles by a small twist in its *own* local coordinates, what twist appears at the point you care about, in *that* point’s coordinates?

The math already has a name for the map that answers this. A rigid transform T carries a small motion from one frame’s coordinates to another’s by conjugation ($T T_\delta T^{-1}$, derived in Appendix D), and the resulting linear map on twists is called the **adjoint representation** of $SE(3)$. For our purposes it is simply the 6×6 matrix that converts “wobble at the interface” into “wobble at the output.”

The adjoint matrix. For a rigid transform $T = (R, t)$, the 6×6 adjoint matrix, in the (v, ω) ordering that matches the twist convention of §1.2, is the block matrix

$$Ad_T = \begin{bmatrix} R & [t]_\times R \\ 0 & R \end{bmatrix}.$$

This maps a twist expressed in T ’s local frame to the equivalent twist expressed in the parent frame, consistent with the right-multiplication convention of §1.2. The off-diagonal block, $[t]_\times R$, is what couples an *angular* input perturbation into a *translational* output effect. It is the linear-algebra statement of the familiar lever-arm effect: a small angular wobble at a joint sweeps out a large lateral displacement by the time it reaches a point far away, $\Delta x \approx L\theta$. (This is physically validated in §2.2.) The coupling term is the single most important fact in the entire theory. Nearly every subtlety in the inverse allocation problem of §1.7 traces back to it.

Propagating sensitivity along a chain. Consider two frames a and b connected by a path of edges. A small perturbation at edge i on that path produces a change in the $a \rightarrow b$

transform equal to the adjoint action, evaluated at the transform from a to the node where edge i exits:

$$J_i = Ad_{T_{a \rightarrow \text{exit}(i)}}.$$

Placing these 6×6 blocks side by side, one per edge on the path, gives a linear map, the Jacobian J , such that to first order

$$\delta_{\text{output}} \approx J \begin{bmatrix} \delta_1 \\ \delta_2 \\ \vdots \end{bmatrix}.$$

This is the adjoint representation of $SE(3)$ acting on its own Lie algebra, specialized to a chain of composed transforms. It involves no new physics beyond the perturbation convention already fixed in §1.2, and it is the entire computational basis for both the variance decomposition of §1.6 and the inverse allocator of §1.7: one piece of linear algebra, reused rather than re-derived for each purpose. (Appendix D derives the adjoint matrix and the per-edge Jacobian blocks from first principles, and spells out what each block of J means.)

One piece of practical advice applies to any derived identity like this. It is easy to make a sign or ordering error in such a derivation, by applying the adjoint from the wrong end of a path, for instance, or by swapping the convention for the translational and rotational blocks. Any implementation should therefore be checked against a numerical finite-difference approximation before anything else is built on top of it. An error of this kind will not usually crash anything. It will silently corrupt every downstream sensitivity computation, which is why it deserves checking on principle rather than after the fact.

1.5 Exact Propagation by Simulation

The linear sensitivity of §1.4 is a first-order approximation around the nominal geometry. It is cheap and exact for infinitesimal perturbations, but for perturbations of realistic size, particularly angular perturbations acting through a long lever arm, the true nonlinear propagation can differ from the linear estimate. The complementary tool is direct simulation: sample a large number of independent trials of every tolerated interface from its specified distribution, compose them exactly (not linearized) along every chain, and examine the resulting empirical distribution of the output pose.

Every trial is an *exact* composition of sampled rigid transforms: full rotation matrices multiplied together, not their linearized twist approximations. The Monte Carlo approach therefore makes no small-angle assumption at the propagation step, even though the *sampling* of each individual tolerance still uses the linearized twist model of §1.2 to turn six numbers into a rigid perturbation. Within the model, simulation is the reference the linear theory of §1.4 is checked against, and it is what the damping loop in §1.7 falls back on when the linear allocation fails to hold up. Keep the qualifier in mind, though: the simulation is only as truthful as the model it samples. It inherits the independence assumption, the

specified distributions, and the rigid-body idealization (§1.8). It removes the linearization error, not the modeling error.

Two properties of a well-posed stochastic simulation are worth stating as general principles:

Reproducibility. For a fixed random seed, a simulation should produce bit-for-bit identical results every time it runs. That sounds unremarkable, but it is essential for any workflow where results are recorded, compared across design iterations, or handed to someone else to reproduce.

Stream independence. More subtly, the sampled behavior of one random variable should not depend on which *other* random variables happen to exist in the model. Adding, removing, or modifying an unrelated tolerance elsewhere in the assembly should never change the sampled draws for a given interface, for a fixed seed and trial count. Without this property, “add one more tolerance to the model” would invisibly perturb every previously validated result. For a tool whose purpose is producing trustworthy, comparable numbers across design iterations, that would be a serious defect.

A third property governs *shared* random variables. If two branches of the kinematic graph both depend on the same upstream interface, that interface’s perturbation must be sampled **once per trial and reused** on both branches, never re-drawn independently for each one. This is not an approximation or a convenience. It is the only way the simulation can capture a real physical effect: two components mounted to the same imperfect reference share that reference’s error, and the shared part can cancel when you ask about their *relative* alignment. Sampling the shared interface twice, independently, would silently destroy the cancellation. The benchmark in §2.3 validates this property physically, and Appendix F proves the cancellation is exact, an algebraic identity rather than a first-order approximation.

1.6 Statistical Characterization of the Propagated Error

A simulation run leaves you with a large cloud of trial poses: fifty thousand slightly different positions and orientations of the frame you care about. This section is about turning that cloud into a handful of numbers an engineer can actually use. Each summary below answers a different practical question, so they are complementary rather than competing.

Before the individual tools, be clear about which kind of question each one answers, because mixing them up is the most common way to misread a tolerance analysis:

Question	Method	What it assumes
“What is the worst that can happen?”	Envelope (min/max box)	Inputs never exceed their bounds
“What happens most of the time?”	Percentiles, confidence ellipsoid	The specified distributions are right

“Which tolerance is driving the error?”	Variance decomposition (Pareto)	First-order linear model
“What fraction of builds will pass?”	Percentile of trials vs. the spec	Distributions right <i>and</i> enough trials to resolve the tails

A worst-case number and a 99th-percentile number can differ by a factor of two or more on the same data, and both are correct answers to different questions. Decide which question the design review is actually asking before quoting either.

First, measure each trial’s error. For translation this is just subtraction: $e_t = t_{trial} - t_{nom}$. For rotation, take the difference rotation $R_{nom}^T R_{trial}$ and extract its axis and angle. The result is a **rotation vector** $\omega = \theta u$, a unit axis u scaled by the angle θ , computed with the matrix logarithm. This is the exact version of the small-angle extraction in §1.2 and reduces to it for small angles. After this step, every trial is six numbers: three translational errors and three rotational ones.

“How bad can it get?” The envelope. The simplest summary is the worst case seen per component: the minimum and maximum of each of the six error numbers across all trials, forming an axis-aligned box. When the worst case is too pessimistic for the decision at hand, percentiles of the same data give a statistical view (for example, the band containing 99% of trials). Both work identically for the absolute error of one frame and for the *relative* pose between any two frames. The relative form is how you answer questions like “how well do these two components align, even though they sit on different branches of the assembly?”

“What shape is the error, really?” The confidence ellipsoid. A translational error cloud is rarely a box. It is usually a squashed, tilted blob: long in the direction some lever arm pushes it, thin elsewhere. An axis-aligned box around such a blob wastes volume and hides the direction that actually matters. A better summary is an ellipsoid aligned with the cloud’s own principal directions (the eigenvectors of its covariance matrix). To size it, give each sampled point a score,

$$r_i^2 = \sum_j \frac{p_{ij}^2}{\sigma_j^2},$$

where p_{ij} is the projection of point i onto principal axis j and σ_j is the spread along that axis. This score, the **Mahalanobis distance**, is just “how many standard deviations out is this point,” measured direction by direction along the cloud’s own axes. The point with the biggest score is the one that sticks out the most, so scale the ellipsoid until exactly that point sits on the surface (multiply every semi-axis by $\sqrt{\max_i r_i^2}$). Every other point is inside by construction. One warning: the shortcut of stretching each axis independently to its own maximum does *not* guarantee enclosure, because a point can be moderately far out along two axes at once and slip through a corner. If you want a statistical ellipsoid instead of a hard bound, use a χ^2 quantile (three degrees of freedom) in place of the sample maximum. Appendix G derives both versions.

“Which way does it tilt?” The rotation cone. Rotational error usually has a preferred axis and spreads radially around it, so the natural summary is a **cone**: a mean axis plus a maximum angular deviation. A box over the three rotation components remains available as a cross-check, but the cone matches the physics better and is the default.

“Whose fault is it?” The Pareto decomposition. The Jacobian of §1.4 makes it possible to split the output variance into per-tolerance contributions, in closed form, like a budget. Each input degree of freedom contributes a variance determined by its distribution and bound b :

$$\text{Var} = \frac{b^2}{3} (\text{uniform}), \text{Var} = \left(\frac{b}{k}\right)^2 (\text{normal, at stated } \sigma\text{-level } k).$$

Weight each contribution by the square of its Jacobian entry, sum, and rank. The result reads like an expense report: this bearing seat owns 60% of the error, that bolt pattern owns 25%, everything else is noise. That is precisely the information you need to decide which component is worth tightening and which is not. Because it comes from the linear model, the ranking is a first-order approximation and is always labeled as such, especially for chains with strong lever-arm coupling (§1.4). (Appendix G derives the variance formulas and the enclosure guarantee above.)

1.7 Inverse Tolerance Allocation

The forward problem asked: given the tolerances, how big is the error? The inverse problem turns that around: given how big the error is *allowed* to be, how loose can each tolerance get? Looser tolerances are easier and cheaper to manufacture, so the goal is to hand out as much looseness as the error budget permits. That makes it an optimization problem, and getting the formulation right is more subtle than it first appears.

Setting up the constraint. Start with what the budget means in the linear model. Row k of the Jacobian (§1.4, Appendix D) tells you how strongly each adjustable tolerance leaks into output direction k . The budget B_k says: all of those leaks, added up at their worst, must fit inside B_k . Written out, with b_{ij} the tolerance bound on edge i , degree of freedom j :

$$\sum_{i,j} |J_{k,6i+j}| b_{ij} \leq B_k.$$

The absolute values are there because each perturbation can independently be positive or negative, so the worst case is when all of them happen to push the same way. Interfaces you have marked as fixed contribute a known amount, which is subtracted from the budget before solving for the rest.

Why the obvious approaches fail. A small example makes both failures visible. Suppose one output budget of 1 mm is shared by two tolerances with equal sensitivity: $b_1 + b_2 \leq 1$.

The first instinct is to scale every tolerance up by one global factor until the tightest budget just binds. The trouble is that one tight requirement then drags down every tolerance in the model, including ones that requirement barely cares about. A tight focus spec in x should

not force a barely-related clamp rotation to be machined ten times finer, but a global scale factor does exactly that.

The second instinct is a linear program: maximize $b_1 + b_2$ directly. But look at the example: $(1,0)$, $(0,1)$, and $(0.5,0.5)$ all give the same objective value of 1, and a linear program always lands on a corner of the feasible region, so it returns $(1,0)$ or $(0,1)$. One tolerance gets everything and the other gets *exactly zero*. A zero tolerance is not conservative. It is a part that cannot be made. And this is not a rare edge case: any nominal rotation between frames couples two degrees of freedom into one budget row and sets up exactly this situation. (Appendix E has the geometric picture.)

The fix: maximize the sum of logarithms.

$$\max_b \sum_{i,j} \log b_{ij} \text{ s.t. } \sum_{i,j} |J_{k,6i+j}| b_{ij} \leq B_k \forall k, b_{ij} \geq \varepsilon,$$

for a small positive floor ε . To see why this works, try it on the example: $\log(0.5) + \log(0.5) \approx -1.39$, while $\log(1.0) + \log(0.001) \approx -6.9$. The log objective strongly prefers the even split. The intuition is that the log cares about *relative* changes: doubling any tolerance adds the same $\log 2$ to the objective, whether that tolerance is currently tight or loose. So it always pays to move budget from a tolerance that has plenty toward one that is starved, until no such move helps, which happens when the budget is spread evenly in the right sense. Three properties follow, each with a practical meaning:

- **No zeros, ever.** As $b \rightarrow 0$, $\log b \rightarrow -\infty$: the objective treats a zero tolerance as infinitely bad. Every free tolerance comes back strictly positive. The log acts as a barrier that walls off the degenerate corners the linear program kept landing on.
- **Looseness goes where sensitivity is low.** At the optimum, a variable's bound is inversely proportional to how strongly the binding budget feels it, $b_j^* \propto 1/a_j$. An input the output barely notices gets a generous bound; a high-leverage input gets a careful one. Equally sensitive inputs split the budget equally. This is exactly the behavior both naive approaches failed to produce, and it comes out of the optimality conditions rather than being programmed in. (Appendix E derives it in a few lines.)
- **The optimum is unique and reproducible.** The objective is concave and the constraints are linear, so this is a convex program: it has exactly one optimum, and any solution the solver finds is that optimum. No multiple-local-minima second-guessing. (Unique within the linear model, that is. Whether the allocation actually meets the budget is still checked against the Monte Carlo model below.)

One practical numerical note. When budgets on different outputs differ by orders of magnitude (a tight translational spec next to a loose angular one), starting every variable from the same initial guess leaves the solver's gradients badly mismatched and it can stall. The cure is to give each variable its own starting point, estimated from the single tightest constraint it participates in, so that every variable begins near its own eventual answer.

Multiple simultaneous targets. The same convex formulation generalizes cleanly to several output targets at once, for instance requiring three optical components to each

satisfy a pairwise alignment specification with the other two, simultaneously. The Jacobians for each target are stacked into one combined constraint system sharing the same set of free variables. An edge that appears on more than one target's path is then automatically constrained by whichever requirement binds tighter, since both constraints must hold in the same feasible region at once. Solving the targets independently and merging the results afterward is not a valid shortcut: it throws away exactly the information the joint problem needs about how one target's remaining slack depends on a shared component.

Reconciling the linear and exact models. Because the allocation above is only a first-order approximation, a candidate solution should always be checked against the exact Monte Carlo model of §1.5 before being trusted. When it fails, most often because of the same angular-to-translational coupling behind the lever-arm effect of §1.4, a natural correction is to iteratively tighten (damp) just the angular bounds by a fixed proportional factor and re-check, repeating until the exact model is satisfied or a fixed number of attempts is exhausted. A subsequent local search (bisection) between the last failing and first passing point then recovers most of the slack that a purely proportional damping step would leave on the table. This loop is a small example of a common and useful pattern: use a cheap linear model to get close quickly, then fall back to an expensive but exact model only to correct the cheap model's blind spots, rather than either optimizing directly against the expensive model or trusting the cheap one unconditionally. As with any iterative numerical correction, each refinement step must be anchored to a fixed reference value rather than to the most recently updated one, or successive iterations can drift from the intended answer.

1.8 Scope: What TolTransform Models, and What It Does Not

Every result in this manual is conditional on the model assumptions. This section collects them in one place so you can judge, before trusting a number, whether your problem actually fits the model. The goal is to prevent misuse, not to hedge.

What TolTransform models well.

- Rigid bodies connected by toleranced interfaces in open (serial or branching) chains: machine frames, optical benches, instrument stacks, gantries, robot-like arms.
- Small perturbations: sub-degree angular play and correspondingly small translations, the regime of precision machine design. Appendix C quantifies how the accuracy degrades as angles grow.
- System-level error budgeting: worst-case and statistical envelopes at any frame, relative alignment between any two connected frames, ranking of error contributors, and inverse allocation against one or more budgets.
- Shared-reference (common-base) analysis: how much of a mounting error cancels out of a relative measurement (§2.3, Appendix F).

What TolTransform does not model.

- *Compliance.* Every body is perfectly rigid. Deflection under load, vibration, and structural flexibility are invisible to the model. The shared-reference cancellation of Appendix F, in particular, is exact only for a base that really is rigid; a base plate that sags between two mounting points needs that sag modeled separately or analyzed with FEA.
- *Thermal effects.* No expansion, no gradients. If thermal growth matters, it must be estimated outside the tool (or bounded and entered by hand as an equivalent tolerance).
- *Correlated errors.* All degrees of freedom, on all interfaces, are statistically independent. A single machining setup that correlates x and y error on one part, or batch effects across parts, are not represented. Where correlation is known to matter, the independent model can be either optimistic or pessimistic, and no general rule says which.
- *Closed kinematic loops.* Four-bar linkages, parallel mechanisms, over-constrained mounts with redundant load paths: none of these fit the tree-structured graph. They need loop-closure constraints the model does not have.
- *Large motions.* A hinge meant to swing, a stage meant to travel: the tool analyzes tolerances about one nominal configuration, not behavior across a range of motion. Analyze each configuration of interest as its own model.
- *GD&T semantics.* Bounds here are plain per-axis limits on a rigid transform. Datum reference frames, material condition modifiers, and feature-of-size rules from ASME Y14.5 have no direct representation, though a careful user can often translate a position or orientation tolerance into an equivalent per-DoF bound.

When to reach for a different tool. Structural deflection or vibration: FEA. Thermal distortion: thermal FEA or hand calculation, then feed the result back in as a bound. Manufacturing process capability and yield economics: SPC and process-capability tools working from measured data. Full GD&T stack-ups on drawings with datum structures: a dedicated 1D/2D stack-up tool, or careful manual translation before using TolTransform. Mechanism motion accuracy across a stroke: multibody kinematics software.

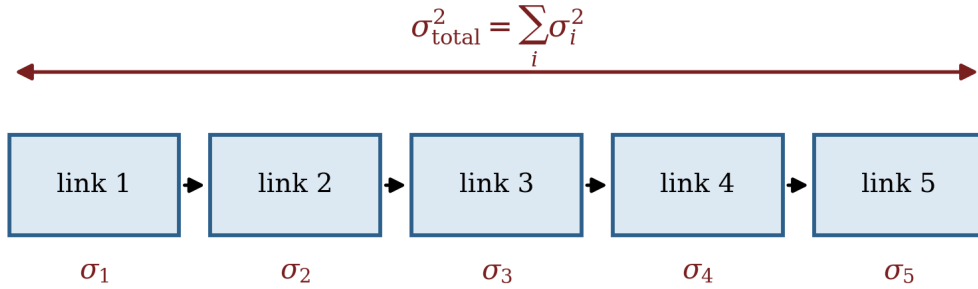
The independence assumption and the small-angle assumption deserve special respect because they fail silently: the tool will happily produce confident-looking numbers either way. The benchmarks of Part 2 verify that the engine computes its model correctly. Only the engineer can verify that the model matches the machine.

Part 2: Validation Against Three Physical Benchmarks

Each of the following checks the theory of Part 1 against a classical, independently derivable analytical result, rather than only against internally hand-calculated test cases. These are the theory's correctness anchors. The inverse allocator of §1.7 is only trustworthy to the extent that the model it optimizes over, and in particular the nonlinear correction it falls back on, has itself been validated this way.

2.1 The Linear Stack-Up Benchmark

Setup. A five-link, purely serial translation chain, with zero nominal rotation between links.



The linear stack-up benchmark: five serial translational links, each with an independent normal tolerance, whose variances must combine by root-sum-square at the end of the chain.

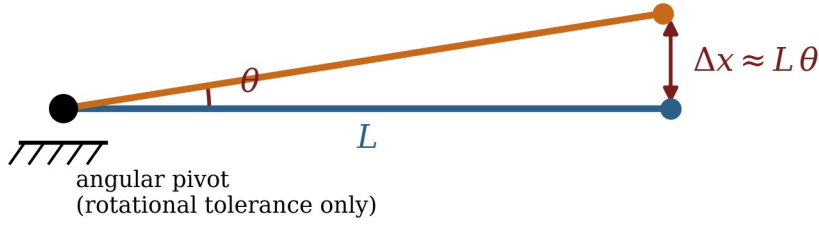
Check. Simulating forward with normal-distribution tolerances on each link's translational degree of freedom, the output variance at the end of the chain must match the classical root-sum-square stack-up formula,

$$\sigma_{\text{total}}^2 = \sum_i \sigma_i^2,$$

which holds exactly, not approximately, for a linear, uncoupled translational stack-up. The match is required to fall within a quantified Monte Carlo sampling-error bound, not an arbitrarily loose tolerance.

2.2 The Sine-Bar Lever-Arm Benchmark

Setup. One angular pivot (a nonzero rotational tolerance, zero translational tolerance) followed by a fixed translational offset of length L .



The sine-bar benchmark: a small angular perturbation θ at the pivot, acting through the arm length L , produces the lateral displacement $\Delta x \approx L\theta$ at the tip.

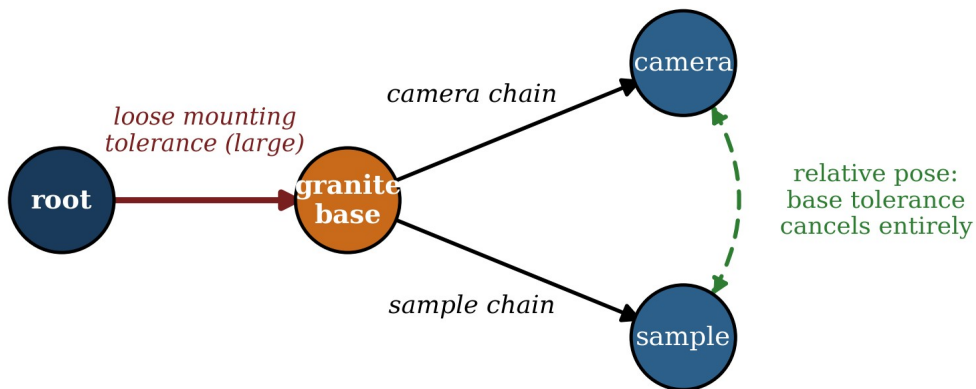
Check. For small angles, the resulting lateral error must match the classic small-angle lever-arm relation,

$$\Delta x \approx L\theta.$$

This directly validates the geometric-leverage effect, the off-diagonal coupling term in the adjoint matrix of §1.4, which is also what motivates the nonlinear correction loop of §1.7. If this benchmark failed, the justification for that correction would collapse with it.

2.3 The Common-Ancestor Cancellation Benchmark

Setup. A shared reference frame (a “granite base”) connected to a fixed root frame through one edge with a large, deliberately loose tolerance, representing real mounting uncertainty. Two further frames, a camera and a sample, each branch independently from the shared base through their own chains.



The common-ancestor cancellation benchmark: the deliberately loose base-mounting tolerance is shared by both branches, and must cancel entirely out of the camera-to-sample relative-pose measurement.

Check. A relative-pose query between the camera and the sample must show that the large base-mounting tolerance cancels out of the relative measurement entirely. The observed camera-to-sample uncertainty must reflect only the camera's and sample's own chain tolerances, no matter how loose the shared base tolerance is.

This formally validates the shared-sampling property of §1.5, and it is the mechanism behind a common engineering question: if two instruments are grounded to the same base plate, how much of that base plate's own mounting uncertainty do you get back for free in their relative alignment?

Part 3: User Guide (Graphical User Interface)

A typical session follows the same arc every time:

1. **Build the topology** in the Graph Editor: create a frame for every rigid body or reference surface, then connect them with edges carrying the nominal transforms (§3.2).
2. **Open up the tolerances** in the Tolerance Editor: new edges start locked at zero, so nothing contributes error until you say so (§3.3).
3. **Run a forward (FK) simulation** from the Run Panel to see how much error accumulates (§3.4).
4. **Read the results:** envelope tables for the numbers, the frame report for the shapes, the Pareto chart to find out which tolerance is responsible (§3.5).
5. If you have a budget to meet rather than a design to check, **run an IK allocation** instead, review the proposed tolerances, and apply them back to the project (§3.4, §3.5).
6. For alignment questions between two specific components, save a **Point-Pair Analysis** so the same question is answered automatically on every future run (§3.6).

Everything the GUI does, the engine can also do headless from a script; the worked examples in Part 4 drive the exact same code. Projects live in a single JSON file, and the file is validated on load, with clear error messages for cycles, dangling frame references, and malformed tolerance entries.

3.1 Overall Layout

The main window hosts five panels as dockable widgets:

- **Left dock:** Graph Editor.
- **Right docks (tabbed):** Tolerance Editor, Run Panel, Results Viewer, Point-Pair Analysis.
- **On-demand window** (View menu, Ctrl+3): 3D Frame Viewer. Not a permanent dock.

File menu: New / Open / Open Recent (last five) / Save / Save As / Exit. A * in the title bar marks unsaved changes, and closing with unsaved changes prompts you to save. Window layout and the recent-files list persist between sessions.

3.2 Graph Editor: Building the Model

The Graph Editor is where you define the topology: Frames (named nodes) and Edges (toleranced transforms between them).

- **Tree view** with two sections, “Frames” and “Edges.” Root frames (in-degree zero) render bold blue with a [ROOT] prefix. Junction frames (more than one outgoing edge) render orange with a [JUNCTION] prefix, so you can see at a glance which frames are shared reference points between multiple chains.
- “+ **Frame**” opens a dialog with a single name field, validated for uniqueness.
- “+ **Edge**” opens a dialog with an edge name, a parent-frame combo, a child-frame combo, and a multi-format nominal-transform entry widget (HTMEntryWidget). A format selector switches between four input pages:
 - XYZ translation + Euler angles (intrinsic ZYX)
 - Raw 4×4 matrix (bottom row locked to [0, 0, 0, 1])
 - Quaternion (wxyz) + XYZ
 - Screw/exponential coordinates (axis, angle, translation along axis)
- Every field is live-validated against the actual transform constructors as you type, so you get real validation errors, not just format matching.
- New edges default to **all-locked-zero tolerance**: safe to simulate immediately without contributing any error, until you deliberately open them up in the Tolerance Editor.
- **Editing an existing edge**: double-click it (or select it and choose “Edit Selected”) to open a pre-populated edit dialog. The edge’s name and nominal transform can be changed in place.
- **Deleting**: deleting a Frame that is still referenced by an edge is blocked with a warning (a referential-integrity guard). Edges can be deleted freely.
- Clicking an edge in the tree auto-selects it in the Tolerance Editor panel, so you can jump straight from “which edge is this” to “what’s its tolerance.”

3.3 Tolerance Editor: Setting Per-DoF Tolerances

Select an edge, either from its own combo or by clicking it in the Graph Editor tree, to see a six-row grid, one row per DoF (dx, dy, dz, rx, ry, rz), each with:

- Distribution combo (uniform / normal)
- Bound spinbox
- σ -level spinbox, greyed out unless normal is selected
- Locked checkbox. A locked DoF is still sampled in FK; it is only excluded from what the inverse allocator is allowed to change.
- An inline per-row validation error label

Every field change writes straight back to the project's schema and is validated live using the same constructor the engine itself uses. GUI validation and engine validation are the same code path, not two separately maintained rule sets.

Bulk-apply group: set a distribution/bound/ σ -level once and apply it either to all six DoF of the currently selected edge, or to all edges in the entire project. Both modes preserve each DoF's individual locked flag.

3.4 Run Panel: Configuring and Executing a Simulation

A mode selector switches between:

FK Verification mode

Set the trial count and a random seed (with a “Randomize” button), then click **Run**. The simulation runs on a background thread, so the UI stays responsive during large runs. Practical guidance: 50,000 trials is a good default for envelope and percentile work; go higher when you care about the far tails (a 99.9th percentile needs enough trials beyond it to be meaningful), and keep the seed fixed while iterating on a design so run-to-run differences reflect your changes, not sampling noise.

IK Allocation mode

A dynamic list of **constraint rows** appears, each with a Frame A / Frame B pair and six target-bound spinboxes (one per DoF). “**Add Constraint**” appends another row, letting you specify multiple simultaneous frame-pair targets, for example mirror1↔mirror2 *and* mirror1↔mirror3 in one run. A max-iterations spinbox (default 30) caps the correction loop. All constraint rows and the max-iteration setting persist with the project file, so re-opening a saved project restores your full IK setup exactly.

3.5 Results Viewer: Reading the Output

Three pages, switched automatically based on which mode you ran:

FK results page

- Frame selector combo → a six-row envelope table (DoF / Min / Max).
- “**Open Frame Report in New Window**” opens a standalone plot window with a 4×3 grid: DX/DY/DZ histograms, XY/XZ/YZ 2D bounding-shape projections, RX/RX/RZ histograms, and a rotation summary (cone + box) spanning the bottom row.
- **Pareto sensitivity section:** pick a Frame A / Frame B pair and click “Compute & Open” to get a standalone bar-chart window ranking which edge/DoF combination dominates the error budget between those two frames. The chart always carries a “first-order linear approximation” annotation (§1.6).

Plots open as standalone OS windows rather than embedded in the dock, so they are never cramped by panel sizes.

How to read an FK result in practice: check the envelope table against your requirement first. If it passes with margin, you are done. If it fails, or passes too tightly, open the Pareto chart before touching any tolerance. It usually shows one or two dominant contributors, and tightening anything else is wasted money. If the envelope looks suspiciously large in one translational direction, look for an angular tolerance upstream acting through a long lever arm (\$1.4); the histograms and 2D projections in the frame report will show the characteristic elongated cloud.

IK results page

- A convergence status label (green = converged, orange = not) with the method name.
- **One results section per constraint pair** (for multi-pair runs), each showing a DoF | Target $\pm$ | Min | Max | Pass? table, with the section's title colored by pass/fail.
- A **corrected-allocation table** (edge $\times$ DoF). Locked DoFs are shown with a dash, and damped cells carry a tooltip showing the baseline (pre-correction) bound for comparison.
- **“Apply Corrected Allocation to Project...”** is enabled only when the run converged. It confirms with a preview of exactly which edges/DoFs will change, then writes the bounds back into the project's actual edge tolerance specs, skipping locked DoFs. This closes the loop: run IK $\rightarrow$ review $\rightarrow$ apply $\rightarrow$ the Tolerance Editor now shows the newly derived, ready-to-use tolerance set as if you had typed it in by hand.

3.6 Point-Pair Analysis Panel

Lets you save **named** (Frame A, Frame B) analyses that persist with the project, independent of any specific run. Useful for tracking the same relative-alignment question (“camera vs. sample”) across repeated simulation runs as you iterate on the design.

- Pick Frame A / Frame B. A connectivity check (pure graph reachability, no simulation needed) warns immediately if the two frames are not connected, or if A equals B.
- Name auto-populates as “FrameA $\rightarrow$ FrameB”. **Save Analysis** adds it to a persistent list (delete via “Delete Selected”).
- After running an FK simulation, selecting a connected pair shows a live **relative-pose envelope table**. This is the panel that answers “what is the actual relative tolerance between these two specific components,” independent of where they sit in the graph.

3.7 3D Frame Viewer (View $\rightarrow$ 3D Frame Viewer, Ctrl+3)

A standalone, on-demand OpenGL window, not a permanent dock, so it does not compete for space with the editing panels. Two modes, toggled by a button:

- **Frames mode (default):** every Frame drawn as an RGB coordinate triad (X = red, Y = green, Z = blue, auto-scaled), with grey lines connecting each frame to its parent: a live 3D wireframe of your nominal geometry. It updates in real time as you edit the graph,

even before you have run a simulation, which makes it the fastest way to catch a wrong rotation or a misplaced frame before wasting a run on it.

- **Point Cloud mode:** after an FK run, pick a Frame from a selector and see its full Monte Carlo trial-pose scatter (up to 500,000+ points at interactive frame rates), colored by depth. This gives a direct visual sense of the actual 3D shape of a frame's uncertainty cloud, complementing the numeric envelope tables and 2D projection plots elsewhere.

Orbit/zoom/pan via mouse drag. A HiDPI-aware viewport fix ensures correct rendering at high display scaling (125%+) on Windows.

Part 4: Brief Use-Case Examples

Four worked, runnable scripts live in `examples/` (python `examples/<script>.py`, no GUI dependency, self-contained printed output). They double as onboarding material and as the source of several regression-test scenarios.

4.1 `single_chain_fk_example.py`: Basic Forward Verification

A 3-edge CNC spindle alignment chain, `world` $\rightarrow$ `spindle_housing` $\rightarrow$ `bearing_seat` $\rightarrow$ `tool_tip`, mixing uniform and normal tolerances, run at 50,000 trials. It demonstrates the core forward-verification workflow (a worst-case envelope plus a percentile table at `tool_tip`) and, concretely, angular error amplification: a 0.001 rad housing-tilt tolerance alone produces roughly 0.150 mm of tip displacement over a 150 mm tool overhang, a direct numeric illustration of the $\Delta x \approx L\theta$ lever-arm effect (§2.2) in a realistic setting.

4.2 `multi_chain_shared_frame_example.py`: Shared-Frame Cancellation

An optical bench with two lenses sharing a common upstream frame, swept across four scenarios:

- Zero lens tolerances: proves shared-ancestor bench errors cancel to floating-point zero ($\sim 2 \times 10^{-13}$ mm) in the relative measurement.
- Translational seat errors only: a direct root-sum-square contribution.
- Rotational seat errors only: reveals the lever-arm amplification. A rotational tolerance of ± 0.001 rad across a 100 mm separation gives ± 0.1 mm relative dy, matching the analytical $L \delta\theta$ prediction exactly.
- Full tolerances: the combined, realistic case, dominated by the rotational lever arm.

This is the runnable demonstration of one of the core engineering decisions TolTransform exists to support, “mitigation verification”: if we ground two components to a shared base, how much absolute error cancels out of their relative alignment?

4.3 `allocation_example.py`: The Inverse Allocator End-to-End

A 3-frame wafer-inspection gantry chain (`wafer_chuck` $\rightarrow$ `stage` $\rightarrow$ `sensor_head`, a 300 mm z-lift plus a 500 mm x lateral arm), in three parts:

- **Unconstrained FK baseline:** shows the raw $\pm$ mm envelope with no allocation applied.
- **Achievable allocation** to a ± 0.1 mm / ± 1 mrad target: prints `converged=True`, the iteration count, and a side-by-side baseline-vs-corrected allocation table with a $\Delta\%$ column, a direct look at how much the correction loop tightened the naive linear answer.

- **A deliberately infeasible target**, with an imprecise stage_mount edge locked to ± 2 mm: prints converged=False, iterations_used=10 (hitting the cap), the achieved-vs-target envelope gap, and three concrete engineering remedies: replace the locked component with a tighter one, restructure the chain, or relax the target. This is a worked example of what a failed allocation run tells you and what to do about it.

4.4 pareto_sensitivity_example.py: Sensitivity Ranking, Component Selection, and Reporting

Demonstrates all three of the “Core Engineering Decisions” the tool exists to support:

- **Sensitivity pinpointing:** a variance-decomposition breakdown answering “which specific edge/DoF is the dominant driver of this error?”
- **Component selection:** upgrades a shoulder joint from ± 3 mrad to ± 1 mrad and directly compares the before/after rankings. This is the concrete “should we pay for the tighter part” workflow.
- **Reporting:** saves a full frame report and both ranking charts as image files, using the same plotting as the GUI’s Results Viewer.

Appendix A: Quick-Reference Glossary

Term	Meaning
HTM	Homogeneous transformation matrix: the canonical 4×4 representation of a rigid-body transform (§1.1).
FK	Forward (tolerance) verification: propagate tolerances forward to see resulting uncertainty. Not robotics forward kinematics in the pose-solving sense, though the analogy is intentional.
IK	Inverse tolerance allocation: back-solve for per-interface tolerance bounds given a target output uncertainty. Not classical inverse kinematics; no joint angles or specific poses are solved for.
DoF	Degree of freedom: one of six independent components of a rigid-body perturbation, $\delta_x, \delta_y, \delta_z, \theta_x, \theta_y, \theta_z$.
Twist (δ)	The six-component element of $se(3)$ describing one small rigid-body perturbation (§1.2).
Adjoint (Ad_T)	The 6×6 linear operator that re-expresses a twist from one rigid frame’s coordinates into another’s (§1.4).
Jacobian (J)	The stacked adjoint blocks giving $\delta_{output} \approx J[\delta_1; \delta_2; \dots]$: the linear sensitivity used for both the variance decomposition and the allocator (§1.4, Appendix D).
Envelope	The per-DoF worst-case min/max bound of a sampled error distribution, for a single frame or a frame pair (§1.6).
Confidence ellipsoid	A Mahalanobis-distance-based enclosure of a translational error cloud along its principal axes (§1.6).
Rotation cone	The preferred summary of rotational uncertainty: a maximum angle plus a mean axis direction (§1.6).
Log-barrier allocation	The convex optimization objective, maximizing $\sum \log b_{ij}$, that allocates the loosest tolerances without ever driving one to zero (§1.7).
Correction loop	The iterative step that checks a linear allocation against the exact Monte Carlo model and tightens angular bounds until it holds (§1.7).

Appendix B: Summary of Modeling Choices

These are the theory's foundational conventions. Changing any one of them would require re-deriving the downstream results that depend on it.

- The small-angle (linearized) approximation for every tolerance twist (§1.2).
- Local-frame, right-composition perturbation convention, $T_{pert} = T_{nom} T_{\delta}$ (§1.2).
- Independent sampling across all six degrees of freedom and across all interfaces. No correlation model (§1.2).
- Kinematic graphs restricted to open, serial chains. No closed loops (§1.3).
- Disconnected sub-assemblies are never silently assumed to share a rigid reference (§1.3).
- The adjoint representation of $SE(3)$ as the sole basis for first-order sensitivity, cross-checked numerically rather than re-derived ad hoc for each use (§1.4).
- Shared upstream interfaces are sampled once per trial and reused, never re-drawn per branch (§1.5).
- The rotation cone, not the axis-aligned box, as the default summary of rotational uncertainty (§1.6).
- The log-barrier objective as the sole basis for inverse allocation, rather than any linear-sum or equal-scaling alternative (§1.7).
- A linear allocation is always checked against, and if necessary corrected against, the exact nonlinear model before being trusted (§1.7).

Appendix C: Why Small Rotations Are Order-Agnostic

If you are familiar with rigid-body kinematics, you may reasonably object to the claim, made in §1.2 and relied on throughout, that the six components of a tolerance twist can be treated as an unordered vector, with rotational perturbations that commute and superpose linearly. After all, one of the first facts everyone learns about rotations is that order matters: rotating a book 90° about x and then 90° about y does not produce the same orientation as the reverse sequence. This appendix works out exactly where the order dependence lives, and shows that for small angles it is a second-order effect, negligible at the same order of approximation as everything else in the linearized theory.

C.1 The Exact Rotation and Its Expansion

A rotation by angle θ about a unit axis u is given exactly by the Rodrigues formula,

$$R(\theta, u) = I + \sin \theta [u]_{\times} + (1 - \cos \theta) [u]_{\times}^2,$$

where $[u]_{\times}$ is the skew-symmetric cross-product matrix of §1.2. Substitute the Taylor expansions $\sin \theta = \theta - \theta^3/6 + \dots$ and $1 - \cos \theta = \theta^2/2 - \dots$ and collect powers of θ :

$$R(\theta, u) = I + \theta [u]_{\times} + \frac{\theta^2}{2} [u]_{\times}^2 + O(\theta^3).$$

Truncating at first order, and writing $[\theta]_{\times} = \theta [u]_{\times}$ for the rotation vector $\theta = \theta u$, gives the small-angle form used throughout Part 1:

$$R \approx I + [\theta]_{\times}.$$

The first neglected term is quadratic in the angle. For a tolerance of 1 mrad, a typical magnitude in precision machine design, the quadratic term is of order $1/2 (10^{-3})^2 = 5 \times 10^{-7}$: four orders of magnitude below the retained term.

C.2 Where Order Dependence Lives: The Commutator

Now compose two small rotations, with rotation vectors α and β , in one order and then the other, keeping *all* the cross terms:

$$R_1 R_2 = (I + [\alpha]_{\times})(I + [\beta]_{\times}) = I + [\alpha]_{\times} + [\beta]_{\times} + [\alpha]_{\times} [\beta]_{\times},$$

$$R_2 R_1 = (I + [\beta]_{\times})(I + [\alpha]_{\times}) = I + [\alpha]_{\times} + [\beta]_{\times} + [\beta]_{\times} [\alpha]_{\times}.$$

The two expressions are identical except for the last term. Their difference is the matrix **commutator**,

$$R_1 R_2 - R_2 R_1 = [\alpha]_{\times} [\beta]_{\times} - [\beta]_{\times} [\alpha]_{\times} = [\alpha \times \beta]_{\times},$$

where the last equality is the standard identity relating the commutator of two skew matrices to the cross product of their vectors. This single term is the entire content of “rotation order matters,” and every entry of it is a *product of two small angles*. If α and β are

each of order ϵ , the order-dependent term is of order ϵ^2 : exactly the size of the terms already discarded in §C.1 when the Rodrigues formula was truncated. To first order, then,

$$R_1 R_2 \approx R_2 R_1 \approx I + [\alpha + \beta]_{\times},$$

which is the statement of superposition: composing small rotations, in any order, is equivalent to a single small rotation whose rotation vector is the *vector sum* of the individual ones.

The classic “rotate the book 90° two ways” demonstration is not evidence against this appendix. It is a demonstration of the commutator at large angles, where $\alpha \times \beta$ is of order one and nothing here applies. The claim is never that rotations commute. The claim is that their failure to commute is quadratic in the angles, and the linearized theory discards quadratic terms everywhere, consistently.

C.3 Consequences and Limits

Three consequences of this derivation carry weight elsewhere in the manual:

Superposition of tolerance DoF (§1.2). The three angular components $(\theta_x, \theta_y, \theta_z)$ of a tolerance twist can be specified as three independent, unordered numbers, because to first order the rotation they generate is $I + [\theta]_{\times}$ regardless of the order in which you imagine applying the components. No “Euler sequence” needs to be declared for a *tolerance*, and the tool asks the user for none.

Linearity of the sensitivity map (§1.4). The Jacobian formalism sums per-edge contributions, $\delta_{output} \approx \sum_i J_i \delta_i$. That sum is meaningful only because the composed effect of many small perturbations is, to first order, their (transformed) vector sum, which is §C.2 applied across edges instead of within one rotation.

An error budget for the approximation itself. The neglected terms are of order ϵ^2 in the tolerance magnitudes. At 1 mrad they contribute at the 10^{-6} to 10^{-7} relative level. Even at a generous 10 mrad ($\approx 0.6^\circ$) they sit near 10^{-4} , still far below typical manufacturing knowledge of the tolerances themselves. This is also why the exact Monte Carlo propagation of §1.5, which composes full rotation matrices and therefore *keeps* every commutator term, agrees with the linear model except in high-leverage geometries. The disagreement it does detect, and the correction loop of §1.7 repairs, comes from the lever-arm coupling of angle into translation, not from rotation-order effects within the angles themselves.

Two boundaries of the argument, stated plainly. First, it applies to *perturbations only*. Nominal transforms in the kinematic graph may contain arbitrarily large rotations, and their order of composition matters fully. That is why the nominal-transform entry formats of §3.2 do pin down an explicit convention (intrinsic ZYX for Euler input), while the tolerance editor of §3.3 needs none. Second, “small” is a quantitative claim, not a qualitative one. The approximation degrades as ϵ^2 , so a joint with genuinely large angular play (several

degrees and up) is outside the theory's domain, which is the position taken in §1.2, now with its error term made explicit.

Appendix D: The Adjoint Matrix and the Sensitivity Jacobian

Section 1.4 states, without proof, that a small perturbation at edge i of a chain shows up at the output transformed by the adjoint matrix Ad_T , with its characteristic off-diagonal lever-arm block, and that stacking these blocks gives a Jacobian. This appendix derives both facts from the composition rule of §1.2 and nothing else, then spells out what the Jacobian is and how each part of the theory uses it.

D.1 Setup: Perturbing One Edge of a Chain

Take a concrete three-edge chain from frame a to frame b :

$$T_{a \rightarrow b} = T_1 T_2 T_3.$$

Now perturb edge 2 alone by a small twist δ , using the right-composition convention of §1.2, so T_2 becomes $T_2 T_\delta(\delta)$. The perturbed chain is

$$\tilde{T}_{a \rightarrow b} = T_1 T_2 T_\delta(\delta) T_3.$$

We want to isolate the effect of δ : how far, and in what direction, did the output move? Compare the perturbed chain to the nominal one by multiplying on the right by the nominal inverse, and watch what cancels:

$$\tilde{T}_{a \rightarrow b} T_{a \rightarrow b}^{-1} = T_1 T_2 T_\delta T_3 (T_1 T_2 T_3)^{-1} = T_1 T_2 T_\delta T_3 T_3^{-1} T_2^{-1} T_1^{-1} = (T_1 T_2) T_\delta (T_1 T_2)^{-1}.$$

The T_3 pair cancels immediately, and what is left has a striking shape. Writing $C = T_1 T_2$, the nominal transform from a to the node where edge 2 exits,

$$\tilde{T}_{a \rightarrow b} T_{a \rightarrow b}^{-1} = C T_\delta(\delta) C^{-1}.$$

The entire effect of the perturbation, seen from frame a , is the small motion T_δ *conjugated* by the fixed transform C . Everything downstream of the perturbed edge has cancelled out, and everything upstream has been absorbed into C . This is the structural reason, visible already, why the sensitivity block for an edge will depend only on the transform from the query frame to that edge's exit node. Nothing in this step used the smallness of δ ; the same cancellation works for any chain length, with C always the product of the edges up to and including the perturbed one.

D.2 Conjugating a Small Motion Gives the Adjoint

The remaining job is to compute $C T_\delta C^{-1}$ to first order, and to see that it is again a small motion, just with a transformed twist. Write the small motion in homogeneous form as $T_\delta = I + \hat{\delta}$, where

$$\hat{\delta} = \begin{bmatrix} [\theta]_\times & d \\ 0^T & 0 \end{bmatrix}$$

collects the translational part $d = (\delta_x, \delta_y, \delta_z)$ and the rotational part $\theta = (\theta_x, \theta_y, \theta_z)$ of the twist. Conjugation then distributes over the sum, exactly, with no approximation:

$$C(I + \hat{\delta})C^{-1} = CC^{-1} + C\hat{\delta}C^{-1} = I + C\hat{\delta}C^{-1}.$$

So the conjugated motion is again of the form “identity plus something small,” and that something is $C\hat{\delta}C^{-1}$. Substitute $C = (R, t)$ and its closed-form inverse from §1.1 and multiply the blocks out:

$$C\hat{\delta}C^{-1} = \begin{bmatrix} R[\theta]_{\times}R^T & Rd - R[\theta]_{\times}R^Tt \\ 0^T & 0 \end{bmatrix}.$$

Two standard identities clean this up. First, $R[\theta]_{\times}R^T = [R\theta]_{\times}$: rotating a cross-product matrix rotates its axis vector. Second, $[v]_{\times}w = -[w]_{\times}v$, which turns the translational block into $Rd - [R\theta]_{\times}t = Rd + [t]_{\times}R\theta$. Reading off the transformed twist, component by component:

$$d' = Rd + [t]_{\times}R\theta, \theta' = R\theta.$$

In words: the rotational part of the perturbation simply gets rotated into the new frame ($\theta' = R\theta$), while the translational part gets rotated *and picks up an extra piece from the rotation acting across the offset t* . Stacking (d', θ') as one 6-vector, the map is a single 6×6 matrix:

$$\begin{bmatrix} d' \\ \theta' \end{bmatrix} = \begin{bmatrix} R & [t]_{\times}R \\ 0 & R \end{bmatrix} \begin{bmatrix} d \\ \theta \end{bmatrix} = Ad_C \begin{bmatrix} d \\ \theta \end{bmatrix}.$$

The adjoint matrix of §1.4 is therefore not a definition but a result: it is the unique linear map produced by conjugating a first-order rigid motion, and its block structure is forced by the algebra. In particular, the lever-arm block $[t]_{\times}R$ was not put in by hand. It fell out of the term $R[\theta]_{\times}R^Tt$: an angular perturbation acting on the *position* of the far frame. Taking magnitudes, $|[t]_{\times}R\theta| \leq |t||\theta|$, with equality when the rotation axis is perpendicular to the offset. That is the familiar $\Delta x \approx L\theta$, now with its geometry (the cross product) and its worst-case alignment made explicit.

D.3 From One Edge to the Jacobian

So far we have one edge’s contribution. Section D.1 showed that a perturbation δ_i at edge i appears at the output as $Ad_{C_i}\delta_i$, where $C_i = T_{a \rightarrow \text{exit}(i)}$ is the nominal transform from the query frame a to edge i ’s exit node. When several edges are perturbed at once, their first-order effects simply add. (Cross terms between two perturbations are products of two small twists, second-order quantities, and are discarded consistently with Appendix C.) The total output twist is therefore

$$\delta_{\text{output}} \approx Ad_{C_1}\delta_1 + Ad_{C_2}\delta_2 + \cdots + Ad_{C_n}\delta_n.$$

The **Jacobian** is nothing more than this sum written as one matrix equation. Lay the n per-edge adjoint blocks side by side into a single $6 \times 6n$ matrix, and stack the n per-edge twists into a single $6n$ -vector:

$$J = \begin{bmatrix} Ad_{C_1} & Ad_{C_2} & \cdots & Ad_{C_n} \end{bmatrix}, \delta_{output} \approx J \begin{bmatrix} \delta_1 \\ \delta_2 \\ \vdots \\ \delta_n \end{bmatrix}.$$

It helps to read J in both directions:

- **A column of J** answers: if this one input DoF wiggles by a unit amount, how does the output pose move? Column block i is edge i 's six columns; within the block, the first three columns are that edge's translational DoF and the last three are its angular DoF, whose columns contain the lever-arm terms.
- **A row of J** answers: which input DoF, across the whole chain, does this one output DoF care about, and how much? Row k lists the sensitivity of output component k to every input component in the model.

Each consumer of J in this manual reads it one of those two ways. The Pareto decomposition of §1.6 works column by column: it squares each entry, weights it by the input's variance, and ranks the totals to find the dominant contributor. The allocator of §1.7 works row by row: each output budget B_k becomes one constraint built from row k . If you have used a robot manipulator Jacobian, this is the same kind of object, with wiggles in place of joint velocities: inputs on one side, a 6-DoF output motion on the other, connected by a matrix that depends only on the nominal geometry.

One more step connects J to the allocator's worst-case constraint and finishes the derivation. For output component k , the contribution of input j is $J_{kj} \delta_j$, and the input is free to take either sign within its bound, $\delta_j \in [-b_j, b_j]$. The largest possible value of the sum over all inputs is reached by choosing every sign so its term aligns with the rest, which gives $\sum_j |J_{kj}| b_j$. This is the triangle inequality, and it is achieved with equality because the inputs are independent: nothing prevents the worst-case combination from occurring. The absolute values in §1.7's constraint are therefore exact for the linear model, not a conservative cushion on top of it.

Appendix E: The Log-Barrier Allocation, Convexity, and the KKT Conditions

Section 1.7 asserts three things about the allocation program: that a linear objective degenerates to zero-tolerance vertices, that the log-sum objective cannot, and that its optimum distributes tolerance in inverse proportion to sensitivity. This appendix proves all three. The whole story fits in one sentence before the details: a linear objective is indifferent between an even split and an all-or-nothing split, so the solver picks a corner; the log objective makes the all-or-nothing split infinitely bad, so the solver settles at the interior point where every tolerance's claim on the budget is equalized.

E.1 Why the Linear Program Produces Zeros

Consider the linear program: maximize $\sum_j b_j$ subject to $Ab \leq B$ and $b \geq 0$, where A is the nonnegative matrix of absolute Jacobian entries. By the fundamental theorem of linear programming, an optimum lies at a *vertex* of the feasible polytope, and a vertex in n variables is defined by n active constraints. When one output constraint couples two variables (one row of A with two nonzero entries a_1, a_2 , which is exactly what a nominal rotation between frames produces), that row supplies only one active constraint. The remaining ones must come from the bounds $b_j = 0$.

The two-variable picture makes this concrete. With one budget row $a_1 b_1 + a_2 b_2 \leq B$, the feasible region is a triangle with vertices $(0,0)$, $(B/a_1, 0)$, and $(0, B/a_2)$. Every nontrivial vertex sets one variable exactly to zero. The objective $b_1 + b_2$ is maximized at whichever axis vertex has the smaller sensitivity, and the interior points, where both tolerances are positive, are never optimal for any linear objective. The only exception is the degenerate tie $a_1 = a_2$, where the optimum is an entire edge of the triangle and the solver returns an arbitrary point of it. The zero allocations observed in practice are this geometry at work, not a solver artifact.

E.2 Convexity of the Log-Sum Program

The barrier program replaces the objective with $f(b) = \sum_j \log b_j$. Each $\log b_j$ is strictly concave on $b_j > 0$ (its second derivative, $-1/b_j^2$, is strictly negative), and a sum of concave functions is concave. The constraints $Ab \leq B$, $b \geq \epsilon$ define an intersection of half-spaces, which is a convex set. Maximizing a concave function over a convex set is a convex program, and for convex programs every local optimum is global. To see why: if $b^\dagger$ were locally but not globally optimal, the straight segment from $b^\dagger$ to a better point b^* would stay feasible (the set is convex), and along it f would sit at or above its chord (the objective is concave), so points arbitrarily close to $b^\dagger$ would already beat it. Contradiction. Strict concavity sharpens the conclusion: the optimum is unique. Within the linear model, then, the solver's answer is reproducible and independent of the starting point (up to numerical conditioning); whether that allocation truly meets the budget is still confirmed against the Monte Carlo model, per §1.7.

E.3 The KKT Conditions and the $1/a_j$ Allocation Rule

Form the Lagrangian with multipliers $\lambda_k \geq 0$ on the output constraints. (Assume the floor $b \geq \varepsilon$ is inactive at the optimum; the barrier guarantees this for any reasonable ε .)

$$L(b, \lambda) = \sum_j \log b_j - \sum_k \lambda_k \left(\sum_j a_{kj} b_j - B_k \right).$$

Setting the derivative with respect to each b_j to zero:

$$\frac{\partial L}{\partial b_j} = \frac{1}{b_j} - \sum_k \lambda_k a_{kj} = 0 \Rightarrow b_j^* = \frac{1}{\sum_k \lambda_k a_{kj}}.$$

Every claim in §1.7 now follows by inspection. A variable's optimal bound is the reciprocal of its multiplier-weighted sensitivity: inputs that the binding constraints are less sensitive to receive looser bounds, continuously and automatically. With a single binding constraint, $b_j^* = 1/(\lambda a_j) \propto 1/a_j$. Substituting back into the active constraint $\sum_j a_j b_j^* = B$ gives $n/\lambda = B$, and hence the closed form

$$b_j^* = \frac{B}{n a_j}$$

for n free variables sharing one budget. Equal sensitivities split the budget equally, and in general each variable receives an equal *share of the constraint*: $a_j b_j^* = B/n$ for every j . That last form is the cleanest statement of what the log objective actually equalizes. It is not the tolerances themselves but their consumption of the binding budget: at the optimum, the marginal cost of tightening any free variable is the same, which is the standard economic reading of KKT multipliers as shadow prices.

A quick numeric check makes the rule tangible. Two free variables, one budget $B=1$, sensitivities $a_1=2$ and $a_2=1$ (variable 1 leaks into the output twice as strongly). The formula gives $b_1^* = 1/(2 \cdot 2) = 0.25$ and $b_2^* = 1/(2 \cdot 1) = 0.5$. The less sensitive variable gets twice the tolerance, the constraint is exactly met ($2 \times 0.25 + 1 \times 0.5 = 1$), and each variable consumes exactly half the budget (0.5 each). Nothing was tuned to make that happen; it is what maximizing the sum of logs does.

Two smaller observations round out the picture. Since $1/b_j = \sum_k \lambda_k a_{kj} > 0$, at least one multiplier must be positive: some constraint is always binding, because otherwise every tolerance could grow without limit. And complementary slackness, $\lambda_k (\sum_j a_{kj} b_j - B_k) = 0$, identifies which output budgets actually limit the design, which is useful diagnostic information in its own right.

Appendix F: Exact Cancellation of Shared-Reference Error

Sections 1.5 and 2.3 claim that when two components hang from a common reference, the reference’s own error cancels *entirely* out of their relative pose. This is a stronger statement than the rest of the linear theory, because it is exact at all orders, and it deserves the short proof that shows why.

F.1 The Identity

Let the shared reference (the “granite base” of §2.3) have world pose W_{base} , *including* whatever sampled perturbation its mounting tolerance produced in a given trial. Let X and Y be the full branch transforms from the base to the camera and to the sample respectively, each including their own perturbations. The two world poses are

$$W_{cam} = W_{base} X, W_{smp} = W_{base} Y,$$

and the relative pose is

$$T_{cam \rightarrow smp} = W_{cam}^{-1} W_{smp} = X^{-1} W_{base}^{-1} W_{base} Y = X^{-1} Y.$$

W_{base} has vanished. Not to first order: identically, by associativity and the group inverse. No small-angle assumption was used anywhere. The base perturbation may be arbitrarily large and the cancellation is still exact, which is why the benchmark of §2.3 can demand cancellation to floating-point precision (about 10^{-13} mm in the worked example of §4.2) rather than to a statistical tolerance.

F.2 What the Proof Requires, and What Breaks It

The one hypothesis doing all the work is that the *same* matrix W_{base} appears in both poses. In a Monte Carlo engine, that is precisely the sample-once-and-reuse rule of §1.5: within one trial, the shared edge’s perturbation must be drawn once and used on both branches. If each branch instead drew its own independent copy, $W_1 \neq W_2$, the relative pose would be $X^{-1} W_1^{-1} W_2 Y$, and the leftover factor $W_1^{-1} W_2$, the composition of two independent draws, would *add* roughly twice the base’s variance to the relative measurement instead of removing it. The gap between the correct and incorrect treatment is not a small bias but the full size of the shared tolerance itself. That is what makes this a correctness property worth validating physically (§2.3) rather than a numerical nicety.

Note also what the proof does *not* say. Only the error of the shared *prefix* cancels. The branch transforms X and Y carry their own perturbations, and those survive in $X^{-1} Y$. The relative measurement is exactly as uncertain as the two branches make it, no more and no less. This is the quantitative content of the engineering intuition that grounding two instruments to one base plate buys back the plate’s error but not the instruments’ own mounting errors.

Finally, remember the model boundary (§1.8). The identity treats the shared base as one rigid body: a single W_{base} for both branches. A real base plate that flexes between the two mounting points, or carries a thermal gradient, is effectively two slightly different

references, and the difference between them does *not* cancel. The cancellation you can bank on is the mounting uncertainty of a base that is stiff compared to your error budget. If the base's own deformation is comparable to the budget, model that deformation explicitly (for example, as a small tolerated edge between two base frames) rather than assuming it away.

Appendix G: Deriving the Statistical Summaries

This appendix derives the closed-form statistical facts used in §1.6 and §2.1: the per-DoF variance formulas, the root-sum-square law they feed, and the enclosure guarantee of the scaled confidence ellipsoid. None of it needs more than a first-year probability course: one integral, one variance-of-a-sum identity, and one careful look at what “inside an ellipsoid” means.

G.1 Variance of a Toleranced Degree of Freedom

Uniform. A uniform distribution on $[-b, b]$ has density $1/(2b)$ and zero mean, so

$$\text{Var} = \int_{-b}^b \frac{x^2}{2b} dx = \frac{1}{2b} \cdot \frac{2b^3}{3} = \frac{b^2}{3}.$$

Normal at a stated σ -level. A normal tolerance specified as “bound b at $k\sigma$ ” means $b = k\sigma$ by definition, so $\sigma = b/k$ and

$$\text{Var} = \left(\frac{b}{k}\right)^2.$$

These two formulas are the entire bridge between a user’s tolerance specification and the variance bookkeeping of the Pareto decomposition (§1.6).

G.2 The Root-Sum-Square Law

For a purely translational serial chain (§2.1), the output position error is *exactly* the sum of the individual link errors. Composing pure translations is vector addition, with no linearization involved. For independent random variables, the variance of a sum is the sum of the variances:

$$\text{Var}\left(\sum_i e_i\right) = \sum_i \text{Var}(e_i) + 2 \sum_{i < j} \text{Cov}(e_i, e_j) = \sum_i \sigma_i^2,$$

with the cross terms vanishing by the independence assumption of §1.2. This is why the benchmark of §2.1 can demand *exact* agreement, up to Monte Carlo sampling error: both the composition and the variance algebra are exact for this geometry, so any discrepancy at all indicates an engine defect. There is no approximation to hide behind. It also locates precisely what the benchmark does and does not test. RSS validates the sampling and composition machinery; the *linearization* is tested separately by §2.2, where an exact result exists for the small-angle limit instead.

G.3 The Enclosure Guarantee of the Scaled Ellipsoid

The idea first, then the algebra. The Mahalanobis distance scores every point by “how many standard deviations out is it,” measured along the cloud’s own axes, so the point with the highest score is unambiguously the one that sticks out the most. Inflate the ellipsoid until exactly that point touches the surface, and everything else must already be inside, because everything else scored lower. That is the whole proof; the rest is writing it down.

Project the N sampled translational errors onto the principal axes of their covariance matrix, giving coordinates p_{ij} (point i , axis j) with per-axis variances σ_j^2 , and give each point its Mahalanobis distance $r_i^2 = \sum_j p_{ij}^2 / \sigma_j^2$. Let $r_{\max}^2 = \max_i r_i^2$ and take the ellipsoid with semi-axes $r_{\max} \sigma_j$. A point i lies inside this ellipsoid exactly when

$$\sum_j \frac{p_{ij}^2}{(r_{\max} \sigma_j)^2} = \frac{r_i^2}{r_{\max}^2} \leq 1,$$

which holds for every sample by the definition of the maximum. Enclosure is guaranteed by construction, with equality for the extreme point, so the ellipsoid is also the smallest of its shape that works.

The tempting alternative, scaling each axis independently by its own maximum projection $m_j = \max_i |p_{ij}|$, carries no such guarantee, and a two-dimensional counterexample makes the failure concrete. Take a point $p = (m_1, m_2)$ that attains the maximum on *both* axes at once. (Nothing prevents this; axis projections are not exclusive.) Its membership test against the per-axis ellipse reads

$$\frac{m_1^2}{m_1^2} + \frac{m_2^2}{m_2^2} = 2 > 1:$$

the point lies well outside the ellipse whose semi-axes it defined. The per-axis construction fails precisely for points that are moderately extreme in several directions at once, and for a roughly ellipsoidal error cloud, that is exactly where the physically interesting worst cases live.

Statistical coverage. For a trivariate normal cloud, r^2 (computed with the true covariance) is a sum of three independent squared standard normals, so it follows a χ^2 distribution with three degrees of freedom. Scaling the axes by $\sqrt{q}$, where q is the χ_3^2 quantile at coverage level p , therefore yields an ellipsoid that contains a given point with probability p . This is the confidence-ellipsoid variant of §1.6: it trades the sample-maximum's hard guarantee for a distributional statement that does not keep growing with N .